

# COMP2054-ADE

## ADE Lec03

### Rules for quick big-Oh proofs

Lecturer: Andrew Parkes

Email: [andrew.parkes@Nottingham.ac.uk](mailto:andrew.parkes@Nottingham.ac.uk)

from <http://www.cs.nott.ac.uk/~pszajp/>

# \*\*\* Definition: "Big-Oh" \*\*\*

For two positive functions  $f(n)$  and  $g(n)$ , of a single variable  $n$ , then

$f(n)$  is  $O(g(n))$  "f is Big-Oh of g"

if and only if there exist constants  $c \geq 0$  and  $n_0 \geq 1$  such that

for all  $n \geq n_0$ .  $f(n) \leq c g(n)$

# Rules for Finding big-Oh

- Reverting to the definition each time is time-consuming and error prone
- Better to develop a set of rules that allow us to very quickly find big-Oh

# “Multiplication Rule” for big-Oh

- Suppose
  - $f_1(n)$  is  $O(g_1(n))$
  - $f_2(n)$  is  $O(g_2(n))$
- Then, from the definition, there exist positive constants  $c_1$   $c_2$   $n_1$   $n_2$  such that
  - $f_1(n) \leq c_1 g_1(n)$  for all  $n \geq n_1$
  - $f_2(n) \leq c_2 g_2(n)$  for all  $n \geq n_2$
- Let  $n_0 = \max(n_1, n_2)$ , then multiplying gives
- $f_1(n) f_2(n) \leq c_1 c_2 g_1(n) g_2(n)$  for all  $n \geq n_0$
- So  $f_1(n) f_2(n)$  is  $O(g_1(n) g_2(n))$

# “Multiplication Rule” for big-Oh

- “Sanity checks”:
  - On doing something new it is usually good to consider some “trivial” cases and verify it works correctly - the “unit tests” of maths 😊
- E.g. What is the big-Oh of  $f(n) = 5n$  ?  
Does it “get rid of the constants”?

Use            5 is  $O(1)$   
                   $n$  is  $O(n)$

Hence  $5 * n$  is  $O(1 * n)$  which is  $O(n)$ ,  
as expected.

# Big-Oh Rules: Drop smaller terms

- Suppose  $f(n) = (1 + h(n))$   
with  $h(n) \rightarrow 0$  as  $n \rightarrow \infty$
- Then  $f(n)$  is  $O(1)$
- (The utility will be to combine with the multiplication rule).

Proof (sketch):

- $h(n) \rightarrow 0$  as  $n \rightarrow \infty$  means that for large enough  $n$  then  $h(n)$  will become arbitrarily close to zero.  
Hence, there exists  $n_0$  such that

$$h(n) \leq 1 \text{ for all } n \geq n_0$$

- So,  $f(n) \leq 2$  for all  $n \geq n_0$
- Hence  $f(n)$  is  $O(1)$  (by using  $c=2$  in the definition)

# Exercise

- What is the big-Oh of  $f(n) = n^2 + n$  ?

# Exercise

- What is the big-Oh of  $f(n) = n^2 + n$  ?
- ANS:

$$f(n) = n^2 * ( 1 + 1/n )$$

$1 + 1/n$  is  $O(1)$  by 'drop small terms'

$n^2$  is trivially  $O(n^2)$

then use multiplication rule,

$$f(n) \text{ is } O(n^2 * 1) = O(n^2)$$



# Big-Oh Rules

- After some thought it should become clear that:
- If  $f(n)$  is a polynomial of degree  $d$  (with positive largest term) then  $f(n)$  is  $O(n^d)$  i.e.,
  1. Drop lower-order terms
  2. Drop constant terms

Note: degree of a polynomial is the highest power  
e.g.  $5n^4 + 3n^2$  is degree 4 and so will be  $O(n^4)$

Proof (sketch):

rewrite the polynomial as  $k * n^d * (1 + \dots)$

# Some Important Functions

These **often** appear in algorithm analysis:

- Constant :  $1$
- Logarithmic :  $\log(n)$
- Linear :  $n$
- “n-log-n” :  $n \log(n)$
- Quadratic :  $n^2$
- Cubic :  $n^3$
- Exponential :  $2^n$

***YOU NEED TO KNOW THESE WELL!!***

***Plot some graphs yourself***

# Recall: “Drop smaller terms”

If  $f(n) = (1 + h(n))$   
with  $h(n) \rightarrow 0$  as  $n \rightarrow \infty$

Then  $f(n)$  is  $O(1)$

- To use this we will need to have some rules for limits of various functions
  - E.g. what do we do with  $(1 + (\log(n) / n))$
  - Does  $h(n) = \log(n) / n$  go to 0 as  $n \rightarrow \infty$ ?
- If have a messy function can try plotting a graph.
- But there are some useful rules:

# Useful limits: exponents vs. powers

- Exponentials grow faster (as  $n \rightarrow \infty$ ) than any power:
- for any fixed  $k > 0$ , and  $b > 1$

$$b^n / n^k \rightarrow \infty$$

and

$$n^k / b^n \rightarrow 0$$

$$\text{E.g. } n^2 / 2^n \rightarrow 0$$

$$\text{E.g. } n^{2000} / 2^{n/100} \rightarrow 0$$

# Exercise

- What is the big Oh of  $f(n) = 2^n + n^2$  ?

**Reminder on Terminology:**

**We say “ $n^2$ ” is a “power law” but we do not call it an “exponential”.**

**An “exponential function of  $n$ ” is one with  $n$  in the exponent, such as  $2^n$  not just having a constant exponent (such as the 2 in  $n^2$ ).**

**The difference between  $n^2$  and  $2^n$  is enormous (try computing them at  $n=1000$ ).**

# Exercise

- What is the big Oh of  $f(n) = 2^n + n^2$  ?
- ANS:

$$f(n) = 2^n * ( 1 + n^2 / 2^n )$$

but  $n^2/2^n \rightarrow 0$  so can drop it.

Hence  $f(n)$  is  $O(2^n)$

# Useful limits: powers vs. logs

- Powers grow faster (as  $n \rightarrow \infty$ ) than any power of a log (assume positive powers)

$$n / (\log n) \rightarrow \infty$$

$$(\log n) / n \rightarrow 0$$

More generally:

$$(\log n)^k / n^{k'} \rightarrow 0 \quad \text{for any fixed } k, k' > 0$$

E.g.  $(\log n)^{100} / n^{0.1} \rightarrow 0$

Though it might not be obvious until large  $n$ .

Roughly:

**“Exponentials dominate powers which dominate logs”**

# Exercise

- What is the big-Oh of  $f(n) = (n \log n) + n^2$  ?



# Exercise

- What is the big Oh of  $f(n) = (n \log n) + n^2$  ?

- ANS:

$$f(n) = n^2 * ( (\log n)/n + 1 )$$

But  $(\log n)/n \rightarrow 0$  so can drop it

Hence  $f(n)$  is  $O(n^2)$

# Exercises (offline):

Give the big-Oh of the following

1.  $3n^3 + 10000n$

2.  $n \log(n) + 2n$

3.  $2^n + n$

If need help, then ask, e.g. in labs/tutorials.

# Exercise (1)

- What is the big-Oh of  $f(n) = n \log (n^2)$

## Exercise (2)

- What is the big-Oh of  $f(n) = n \log (n^2)$

ANS:  $O(n \log n)$

because  $\log(n^2) = 2 \log(n)$

## Exercise (3)

- What is the big-Oh of  $f(n) = n \log (n^2)$

ANS:  $O(n \log n)$  as  $\log(n^2) = 2 \log(n)$

- Is  $f(n) = n (\log n)^2$  also  $O(n \log n)$ ?

## Exercise (3)

- What is the big-Oh of  $f(n) = n \log (n^2)$   
ANS:  $O(n \log n)$  as  $\log(n^2) = 2 \log(n)$
- Is  $f(n) = n (\log n)^2$  also  $O(n \log n)$ ?  
ANS: NO! Because
  - it has an extra factor of  $\log n$ , and
  - $\log(n)$  is not  $O(1)$
- Lessons: be careful of the placement of parentheses, and do not think that  $\log(n)$  tends to a constant, it is not  $O(1)$  !

# Big-Oh Conventions

## Conventions:

- Use the smallest (slowest growing) 'reasonable' possible class of functions
  - Say " $2n$  is  $O(n)$ " instead of " $2n$  is  $O(n^2)$ "
- Use the simplest expression of the class
  - Say " $3n+5$  is  $O(n)$ " instead of " $3n+5$  is  $O(3n)$ "
- In both cases the second version is formally correct, but it would not be said without good reason

# Usage of 'O' in practice

- From the definition is true that for any  $f(n)$  that  $f(n)$  is  $O(f(n))$
- But such an answer is inappropriate because conventions say that we want a 'useful' answer, not a trivial one.
- If asked for the 'big-Oh' then want the 'tightest nice function' - this is defined by community standards, and to convey the maximum (practical) information
- Warning: This is a convention and not directly part of the definition – and so causes a lot of confusion.
- **It will be expected on assessments that you use the conventions when appropriate**



# “Algorithm” or “Problem”

- We will see that the big-Oh for a solution to a particular problem depend on the algorithm used.
- Point: picking an appropriate algorithm is needed in order to get good behaviour, and the big-Oh helps this
- It would often be nice to know, for a particular problem, the “best possible big-Oh”, but
  - such lower-bounds are very rarely known
  - it is very hard to do such analyses
  - there are many problems where the entire CS community has entirely failed to make progress, see Millennium prize on **P** vs. **NP**

# Exercise

- What is the big Oh of  $f(n) = 2^{n/100} + n^{200}$  ?
- ANS:

$$f(n) = 2^{n/100} * ( 1 + n^{200} / 2^{n/100} )$$

but  $n^{200}/2^{n/100} \rightarrow 0$  so can drop it.

Hence  $f(n)$  is  $O(2^{n/100})$ .

But in practice, the large power law will dominate until  $n$  is very large.

“Big-Oh” is not a panacea – need to know when it might be misleading

# Exercise

Consider an (extreme) example

- Algorithm A has runtime  $f_1(n) = n^{20}$
- Algorithm B has runtime  $f_2(n) = 2^{n/10}$

Which is the best?

- If only look at Big-Oh, then A wins
- But at  $n=100$  we have
  - $f_1(100) = 100^{20} = 10^{40}$
  - $f_2(100) = 2^{10} = 1024 \sim 10^3$
- Now B clearly wins.
  - Less extreme versions of this happen e.g. “simplex algorithm” (used in optimisation) is exponential in worst case, but often better in practice.
- “Big-Oh” is not a panacea!

# Reminder: Big-Oh as a “Set”

One can think of  $O(n)$  as

“the set of all functions whose growth is no worse than linear for sufficiently large  $n$ ”

Hence, it can be thought of as the (infinite) set  
 $\{1, 2, \dots, \log n, 2 \log n, \dots, n, 2n, 3n, \dots, n+1, n+2, \dots\}$

Then “ $2n+3$  is  $O(n)$ ” is just the statement that the function  $2n+3$  is in this set, i.e.  $2n+3 \in O(n)$

# Big-Oh as a “Set”

Although many sources do it, personally,  
I recommend against writing  $n = O(n)$ , because

- $n = O(n^2)$
- $n = O(n)$

should lead to  $O(n) = O(n^2)$  which is wrong!

Though, note that  $O(n) \subset O(n^2)$ .

That is, if  $f(n) \in O(n)$  then  $f(n) \in O(n^2)$   
(though not the converse)

The (subset) inclusion is strict

e.g. consider  $n^2$  which is in  $O(n^2)$  but not in  $O(n)$

# Big-Oh: Usage for Algorithms

Big-Oh definitions themselves, in pure sense, are just ways of classifying functions and not algorithms

Their usage for runtimes of algorithms has further choices. One can use big-Oh to describe any of:

- Worst case runtime,  $w(n)$ , at each value of  $n$
- Best case runtime,  $b(n)$ , at each value of  $n$
- Average case runtime,  $b(n)$ , at each value of  $n$
- Etc.

If simply say “algorithm X is  $O(\cdot)$ ” then the usual convention is that it will refer to the worst case.

# Big-Oh: Usage for Algorithms

- Worst case runtime,  $w(n)$ , at each value of  $n$
- Best case runtime,  $b(n)$ , at each value of  $n$
- Average case runtime,  $a(n)$ , at each value of  $n$

If simply say “algorithm X is  $O(\cdot)$ ” then the usual convention is that it will refer to the worst case.

But have the freedom to say:

For algorithm X,

- the worst case (over all possible inputs) is  $O(n^3)$
- the average case, given inputs generated uniformly at random, is  $O(n^2)$
- the best case (over all possible inputs) is  $O(n)$ .

E.g. will see:

worst case of quicksort is  $O(n^2)$  but average case is  $O(n \log n)$

# Big-Oh: Usage for Algorithms

Consider:

“Merge-sort is  $O(n \log n)$ ”

expands into:

“If running merge-sort on  $n$  integers, then the worst-case run-time over all possible inputs, is a member of the set of functions that, is no worse than some fixed constant times  $n \log(n)$  for all values of  $n$  that are at least some fixed value.”

Which follows the definition, but the convention might also imply it is the best, and so we might add that the  $O$  is a “tight bound” or “cannot be improved” because:

“there will be some inputs for which the runtime is as bad as this.”



# Asymptotic Algorithm Analysis in practice

- The asymptotic analysis of an algorithm determines the running time in big-Oh notation
- To perform the asymptotic analysis
  - We find the (worst-case, etc.) number of primitive operations executed as a function of the input size
  - We express this function with big-Oh notation
- Example:
  - We determine that algorithm A1 executes at most  $6n^2 + 3n - 4$  primitive operations
  - We say that algorithm A1 “runs in  $O(n^2)$  time”
- In practice:

Since constant factors and lower-order terms are eventually dropped anyhow, if we only want the big-Oh behaviour, then we could decide to disregard them when counting primitive operations – but be careful not to drop the important terms!

# Summary & Expectations

Included, (but not limited to)

- Know the definition of big-Oh well!
  - Be able to apply it, and prove results on big-Oh of simple functions
  - Know how to manipulate
    - Sums
    - Productsof functions, and be able to prove if needed
- Know the “conventions of usage” and also the advanced usages  
e.g. “best case is  $O(\cdot)$ ” or “ $n^{O(1)}$ ”

# Next: Definition: “Big-Omega”

For two positive functions  $f(n)$  and  $g(n)$ , of a single variable  $n$ , then

$f(n)$  is  $\Omega ( g(n) )$     “ $f$  is Big-Omega of  $g$ ”

if and only if there exist constants  $c \geq 0$  and  $n_0 \geq 1$  such that

for all  $n \geq n_0$ .     $f(n) \geq c g(n)$

Spot the difference!?